In [1]:

```
import pandas as pd

from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB, BernoulliNB
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)

# Step 2: Load the 20 Newsgroups dataset
print("Loading the 20 Newsgroups dataset...")

categories = [
    "comp.graphics",
    "sci.space",
    "rec.sport.baseball",
    "talk.politics.misc"
]

train_data = fetch_20newsgroups(
    subset="train",
    categories=categories,
    remove=("headers", "footers", "quotes"),
    random_state=42
)

test_data = fetch_20newsgroups(
    subset="test",
    categories=categories,
    remove=("headers", "footers", "quotes"),
    random_state=42
)

X_train_text = train_data.data
y_train = train_data.target

X_test_text = test_data.data
y_test = test_data.target

print("Number of training documents:", len(X_train_text))
print("Number of testing documents:", len(X_test_text))
print("Topics:", train_data.target_names)

# Step 3: Convert text documents into numerical word-count features
vectorizer = CountVectorizer(
    stop_words="english",
    max_features=10000,
    min_df=2
)

X_train = vectorizer.fit_transform(X_train_text)
X_test = vectorizer.transform(X_test_text)

print("\nTraining feature matrix:", X_train.shape)
```

```
print("Testing feature matrix:", X_test.shape)

# Step 4: Implement Multinomial Naive Bayes
multinomial_model = MultinomialNB(alpha=1.0)

multinomial_model.fit(X_train, y_train)

multinomial_predictions = multinomial_model.predict(X_test)

# Step 5: Implement Bernoulli Naive Bayes
# Convert word counts into binary word-presence features
X_train_binary = (X_train > 0).astype(int)
X_test_binary = (X_test > 0).astype(int)

bernoulli_model = BernoulliNB(alpha=1.0)

bernoulli_model.fit(X_train_binary, y_train)

bernoulli_predictions = bernoulli_model.predict(X_test_binary)

# Step 6: Evaluate both models
def evaluate_model(model_name, y_true, y_pred):
    return {
        "Model": model_name,
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(
            y_true, y_pred, average="weighted", zero_division=0
        ),
        "Recall": recall_score(
            y_true, y_pred, average="weighted", zero_division=0
        ),
        "F1-Score": f1_score(
            y_true, y_pred, average="weighted", zero_division=0
        )
    }

results = [
    evaluate_model(
        "Multinomial Naive Bayes",
        y_test,
        multinomial_predictions
    ),
    evaluate_model(
        "Bernoulli Naive Bayes",
        y_test,
        bernoulli_predictions
    )
]

results_df = pd.DataFrame(results)

print("\n========== MODEL COMPARISON ==========")
print(results_df.round(4).to_string(index=False))

# Step 7: Display detailed classification reports
print("\n========== MULTINOMIAL NAIVE BAYES REPORT ==========")
print(
    classification_report(
        y_test,
        multinomial_predictions,
```

```
        labels=range(len(train_data.target_names)),
        target_names=train_data.target_names,
        zero_division=0
    )
)

print("\n========== BERNOULLI NAIVE BAYES REPORT ==========")
print(
    classification_report(
        y_test,
        bernoulli_predictions,
        labels=range(len(train_data.target_names)),
        target_names=train_data.target_names,
        zero_division=0
    )
)

# Step 8: Identify the model with the highest weighted F1-score
best_model = results_df.loc[
    results_df["F1-Score"].idxmax(), "Model"
]

print("\nBest model based on weighted F1-score:", best_model)
```

```
Loading the 20 Newsgroups dataset...
Number of training documents: 2239
Number of testing documents: 1490
Topics: ['comp.graphics', 'rec.sport.baseball', 'sci.space', 'talk.politic
s.misc']

Training feature matrix: (2239, 10000)
Testing feature matrix: (1490, 10000)

========== MODEL COMPARISON ==========
                  Model  Accuracy  Precision  Recall  F1-Score
Multinomial Naive Bayes    0.8718     0.8760  0.8718    0.8718
  Bernoulli Naive Bayes    0.7134     0.7648  0.7134    0.7036

========== MULTINOMIAL NAIVE BAYES REPORT ==========
                    precision    recall  f1-score   support

     comp.graphics       0.91      0.90      0.91       389
rec.sport.baseball       0.87      0.91      0.89       397
         sci.space       0.92      0.79      0.85       394
talk.politics.misc       0.78      0.89      0.83       310

          accuracy                           0.87      1490
         macro avg       0.87      0.87      0.87      1490
      weighted avg       0.88      0.87      0.87      1490


========== BERNOULLI NAIVE BAYES REPORT ==========
                    precision    recall  f1-score   support

     comp.graphics       0.77      0.87      0.82       389
rec.sport.baseball       0.58      0.94      0.72       397
         sci.space       0.91      0.52      0.66       394
talk.politics.misc       0.82      0.48      0.61       310

          accuracy                           0.71      1490
         macro avg       0.77      0.70      0.70      1490
      weighted avg       0.76      0.71      0.70      1490


Best model based on weighted F1-score: Multinomial Naive Bayes
```

In [ ]: